SC2203 Automata, Computability & Complexity
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Alphabets, Strings and Languages

"What is a language, formally?" — Before automata can recognise anything, we need a precise universe: symbols strung into words, words collected into languages, and operations that build new languages from old.

From zero: we build here, no prior theory needed. We assume only sets and functions. Every notion—alphabet, string, language, Kleene star—is motivated by a picture, then defined, then exercised. No automata, grammars, or computability is assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) define alphabet, string, empty string and length, and count strings over an alphabet;

(L2) distinguish string operations (concatenation, reversal, substring) and prove their basic laws;

(L3) define language as a set of strings and apply Boolean and concatenation operations to languages;

(L4) define Kleene star, plus, and closure properties, and compute small examples;

(L5) recognise lexicographic and shortlex orderings and prove elementary cardinality results.

1.1 Alphabets and Strings

A *symbol* is an indivisible mark—$\mathtt{a}$, $\mathtt{0}$, #. An *alphabet* Σ is a finite non-empty set of symbols. Examples: $\Sigma = \{0, 1\}$ (binary), $\Sigma = \{a, b, \ldots, z\}$, $\Sigma = \{\mathtt{a}, \mathtt{b}\}$.

Definition 1.1 (String, empty string, length)**.** A *string* (word) over Σ is a finite sequence $w = a_1 a_2 \cdots a_n$ with each $a_i \in \Sigma$, $n \geq 0$. When $n = 0$ we get the *empty string* ε. The *length* $|w| = n$.

We write Σ^* for "all strings" (defined formally below). $|\Sigma| = k$ means alphabet size k.

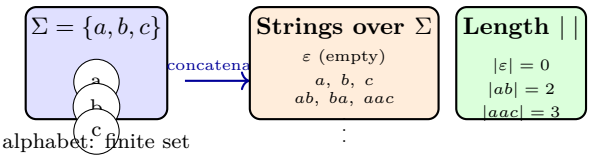


Figure 1.1: Alphabet Σ is a finite symbol set; strings are finite sequences over Σ; $|w|$ counts symbols.

Definition 1.2 (String operations)**.** For $x = a_1 \cdots a_m$, $y = b_1 \cdots b_n$: *concatenation* $xy = a_1 \cdots a_m b_1 \cdots b_n$; *reversal* $x^R = a_m \cdots a_1$ ($\varepsilon^R = \varepsilon$); z is a *substring* of w if $w = xzy$ for some x, y; prefix / suffix are x / y when $y = \varepsilon$ / $x = \varepsilon$.

Example 1.1 (Laws by picture then proof)**.** $|xy| = |x| + |y|$, $(xy)z = x(yz)$, $\varepsilon w = w\varepsilon = w$, and $(w^R)^R = w$. Intuitively, concatenation glues two rows of symbols. Formally, $(xy)z$ and $x(yz)$ are the same symbol sequence $a_1 \cdots a_m b_1 \cdots b_n c_1 \cdots c_p$. Hence concatenation is associative with identity ε—a monoid.

Counting: if $|\Sigma| = k$, there are k^n strings of length n, and $\sum_{i=0}^{n} k^i$ of length $\leq n$. For binary $\Sigma = \{0, 1\}$, 2^n strings of length n—the familiar exponential. Thus even for $k = 1$, say $\Sigma = \{a\}$, there is exactly one string per length: $\Sigma^* = \{\varepsilon, a, aa, \dots\}$ in bijection with $\mathbb{N}$ via $n \mapsto a^n$.

Example 1.2 (Small enumeration)**.** Binary $\Sigma = \{0, 1\}$ with shortlex: length 0: ε (1 string); length 1: $0, 1$ (2); length 2: $00, 01, 10, 11$ (4); length ≤ 2: 7 strings. In general $|\Sigma^{\leq n}| = (k^{n+1} - 1)/(k - 1)$ for $k > 1$ (geometric sum). For $\Sigma = \{a, b, c\}$, $|\Sigma^{\leq 2}| = 1 + 3 + 9 = 13$.

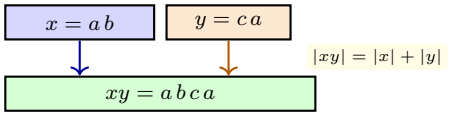


Figure 1.2: Concatenation glues symbol rows; length adds; ε is the empty row.

1.2 Languages: Sets of Strings

A *language* over Σ is any set $L \subseteq \Sigma^*$—finite or infinite. This set viewpoint lets us reuse Boolean operations.

Definition 1.3 (Language operations)**.** For $L, L_1, L_2 \subseteq \Sigma^*$: complement $\overline{L} = \Sigma^* \setminus L$; union $L_1 \cup L_2$, intersection $L_1 \cap L_2$; concatenation $L_1 L_2 = \{xy \mid x \in L_1,\, y \in L_2\}$; $L^R = \{w^R \mid w \in L\}$; power $L^0 = \{\varepsilon\}$, $L^{n+1} = L^n L$.

Example 1.3 (Finite vs. infinite)**.** Over $\Sigma = \{0, 1\}$: $L_1 = \{01, 10\}$ (finite, $|L_1| = 2$); $L_2 = \{0^n 1^n \mid n \geq 0\} = \{\varepsilon, 01, 0011, \dots\}$ (infinite, one string per n); $L_3 = \Sigma^*$ (all strings); $L_4 = \emptyset$ (no strings) vs. $\{\varepsilon\}$ (one string, the empty one)—a classic confusion.

Remark 1.1 (Empty confusion). $\emptyset \neq \{\varepsilon\}$: the first has 0 strings, the second has 1. Moreover $\emptyset L = \emptyset$ but $\{\varepsilon\}L = L$. Concatenating "no choice" yields no word; concatenating "choose nothing" leaves L unchanged.

1.3 Kleene Star and Closure

We close a language under concatenation repeatedly.

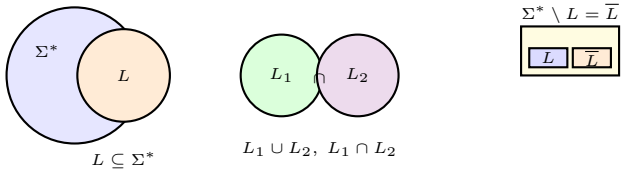


Figure 1.3: Languages sit inside Σ^*; Boolean operations are set operations; complement is relative to Σ^*.

Definition 1.4 (Kleene star and plus)**.** $L^* = \bigcup_{n\geq 0} L^n = \{\varepsilon\} \cup L \cup LL \cup LLL \cup \cdots$. $L^+ = LL^* = L^*L = \bigcup_{n\geq 1} L^n = L^* \setminus \{\varepsilon\}$ if $\varepsilon \notin L$; in general $L^* = L^+ \cup \{\varepsilon\}$. Σ^* is the special case $L = \Sigma$ viewed as $|\Sigma|$ one-symbol strings.

Intuition: L^* is "zero or more L-blocks glued"; L^+ is "one or more". For $L = \{a\}$: $L^* = \{\varepsilon, a, aa, aaa, \dots\} = a^*$ in regex notation; $(L^*)^* = L^*$ (idempotent).

Theorem 1.1 (Algebraic laws)**.** For all L: $(L^*)^* = L^*$, $\emptyset^* = \{\varepsilon\}$, $\{\varepsilon\}^* = \{\varepsilon\}$, and $L^* \supseteq L$. If $L_1 \subseteq L_2$ then $L_1^* \subseteq L_2^*$. However $(L_1 \cup L_2)^* \neq L_1^* \cup L_2^*$ in general.

Proof. $(L^*)^*$ concatenates blocks each itself a concatenation of L-blocks—still a concatenation of L-blocks. Counterexample for union: $L_1 = \{a\}, L_2 = \{b\}$ gives $ab \in (L_1 \cup L_2)^*$ but $ab \notin L_1^* \cup L_2^* = \{a\}^* \cup \{b\}^*$. □ □

Example 1.4 (Counting Σ^*)**.** $|\Sigma^n| = |\Sigma|^n$. Hence Σ^* is countably infinite and enumerable in *shortlex* (by length then dictionary): $\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots$ for binary. Lexicographic alone is not a well-order for listing Σ^* without grouping by length.

1.4 Orderings and Cardinality of Languages

Intuition: list all strings shortest first; within each length, dictionary order. This *shortlex* is the canonical enumeration that proves Σ^* is countable—crucial for diagonalisation in Ch. 7.

Definition 1.5 (Lexicographic vs. shortlex)**.** Fix an order on Σ (e.g. $0 < 1$). *Lexicographic* ($\leq_{\text{lex}}$) compares at the first differing position (and ε is smallest; a prefix is smaller than its extension). *Shortlex* ($\leq_{\text{sl}}$): $x \leq_{\text{sl}} y$ if $|x| < |y|$ or $|x| = |y|$ and $x \leq_{\text{lex}} y$.

Theorem 1.2 (Countability)**.** For finite Σ, Σ^* is countable; the set of all languages $\mathcal{P}(\Sigma^*)$ is uncountable. Shortlex gives an explicit bijection $\Sigma^* \to \mathbb{N}$.

Proof. Shortlex lists strings by increasing length; each length class has k^n strings ($k = |\Sigma|$), finite, so every string appears at a finite position—a bijection $f(w) =$ rank in shortlex. Uncountability of $\mathcal{P}(\Sigma^*)$ follows from Cantor: if Σ^* were countable, its power set would be strictly larger, i.e. $|\mathcal{P}(\mathbb{N})| > |\mathbb{N}|$. □ □

Example 1.5 (Why shortlex not plain lex?)**.** Plain lex on $\{0,1\}^*$ gives $\varepsilon < 0 < 00 < 000 < \cdots$—infinitely many strings before 1, so 1 has no finite rank. Shortlex avoids this by finishing each length:

$$\varepsilon \text{ (0 of len 0)}; \quad 0, 1 \text{ (2 of len 1)}; \quad 00, 01, 10, 11 \text{ (4 of len 2)}; \quad \dots$$

1.5 Chapter Notes

Finite alphabets, strings, and Kleene closure were formalised by Kleene (1956) and systematised by Hopcroft–Ullman. The $\emptyset$ vs. $\{\varepsilon\}$ pitfall recurs in every later chapter; test it whenever a construction claims "no word". Shortlex enumeration underlies dovetailing in Ch. 6–7 and the existence of non-recursively-enumerable languages.

1.6 Exercises

E1.1 **Empty vs. empty language.** Give Σ, then compute $|\{\varepsilon\}|$, $|\emptyset|$, $|\{\varepsilon\}^*|$, $|\emptyset^*|$, and $\emptyset\{a,b\}^*$. Explain why $\emptyset L = \emptyset \neq L$ in words.

E1.2 **Counting.** For $\Sigma = \{a,b,c\}$ count strings of length exactly 3 and ≤ 3. Prove $|\Sigma^n| = |\Sigma|^n$ by induction on n, using $|\Sigma^{n+1}| = |\Sigma| \cdot |\Sigma^n|$.

E1.3 **Star laws.** Prove $(L^*)^* = L^*$ and find L_1, L_2 with $(L_1 L_2)^* \neq L_1^* L_2^*$. Prove or disprove: $(L^R)^* = (L^*)^R$.

E1.4 **Reversal.** Prove $(xy)^R = y^R x^R$ and by induction that $(w^n)^R = (w^R)^n$, then deduce $(L^n)^R = (L^R)^n$ and the previous exercise.

E1.5 **Lexicographic pathology.** List $\Sigma^* = \{0,1\}^*$ first in lexicographic $(0 < 1)$ order: $\varepsilon, 0, 00, 000, \ldots$. Why does 1 never appear? Define shortlex and prove it enumerates Σ^* as $\mathbb{N}$.

Chapter 2

Regular Languages: DFAs, NFAs and Regular Expressions

"A finite automaton is the simplest computer." — It reads left to right, remembers only a finite state, and decides membership. From this humble model flow NFAs, subset construction, and regexes—all equally powerful.

From zero: we build here, no automata assumed. We define DFA from scratch—states, transitions, extended delta—with pictures before formalism. NFAs and ε-moves are motivated by "guessing". No prior complexity is assumed; only Ch. 1 strings/languages.

Learning Outcomes

After this chapter you will be able to:

(L1) define DFA formally and compute its language via $\hat{\delta}$;

(L2) define NFA/ε-NFA and explain nondeterminism as existential choice;

(L3) convert NFA $\to$ DFA via subset construction and argue correctness;

(L4) define regular expressions and their language $L(R)$;

(L5) prove Kleene's theorem: regex $\equiv$ NFA $\equiv$ DFA in expressive power.

2.1 Deterministic Finite Automata

Intuition: a turnstile that clicks through states as symbols arrive; whether you end in a green (accept) state decides membership. Only the current state matters—finite memory.

Definition 2.1 (DFA)**.** A *deterministic finite automaton* is $M = (Q, \Sigma, \delta, q_0, F)$ where Q finite states, Σ alphabet, $\delta : Q \times \Sigma \to Q$, $q_0 \in Q$ start, $F \subseteq Q$ accept. Extend $\hat{\delta} : Q \times \Sigma^* \to Q$ by $\hat{\delta}(q, \varepsilon) = q$, $\hat{\delta}(q, wa) =$

$\delta(\hat{\delta}(q,w),a)$. Language $L(M) = \{w \mid \hat{\delta}(q_0,w) \in F\}$.

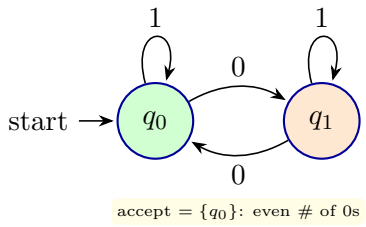


Figure 2.1: DFA for $L = \{w \in \{0,1\}^* \mid \#_0(w) \text{ even}\}$: state remembers parity of 0s; 1s loop.

Example 2.1 (Running a DFA)**.** M above on $w = 010$: $\hat{\delta}(q_0,0) = q_1$, $\hat{\delta}(q_0,01) = q_1$, $\hat{\delta}(q_0,010) = q_0 \in F$ so $010 \in L$. On $w = 0$: ends in $q_1 \notin F$, reject. The table of δ is 2×2 entries; $\hat{\delta}$ is just iterated lookup.

Example 2.2 (Code view: DFA simulator)**.**

```
def accepts(dfa, w):
    q = dfa.q0
    for a in w:           # deterministic: one next state
        q = dfa.delta[(q,a)]
    return q in dfa.F     # accept iff end in F
# dfa for even-zeros: Q={0,1}, delta={(0,'0'):1,(0,'1'):0,(1,'0'):0,(1,'1'):1}
```

Every DFA is a pure function $\Sigma^* \to Q$; no branching, $O(|w|)$ time.

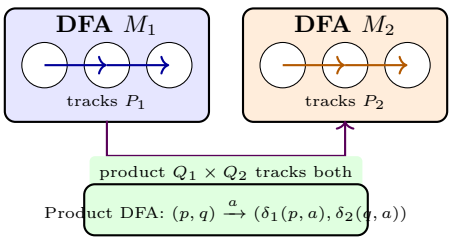


Figure 2.2: Closure: run two DFAs in parallel via product states (p,q); accept according to $F_1 \times F_2$ ($\cap$), union for $\cup$, complement by flipping F.

Closure under Boolean ops follows: $\overline{L(M)}$ flips F to $Q \backslash F$; $L(M_1) \cap L(M_2)$ is the product DFA with $F = F_1 \times F_2$; union uses $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.

2.2 Nondeterminism and ε-Transitions

NFA *guesses* among several next states (or ε-moves without consuming input); w accepted if *some* guess leads to accept. Surprisingly, no more powerful than DFA.

Definition 2.2 (NFA and ε-NFA)**.** $N = (Q, \Sigma, \delta, q_0, F)$ with $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. ε-closure $\textsc{Eclose}(S)$ = all states reachable via ε-only paths from S. Then $\hat{\delta}(q,\varepsilon) = \textsc{Eclose}(\{q\})$, $\hat{\delta}(q,wa) = \textsc{Eclose}(\bigcup_{p \in \hat{\delta}(q,w)} \delta(p,a))$. $w \in L(N)$ iff $\hat{\delta}(q_0,w) \cap F \neq \emptyset$.

Example 2.3 (NFA for "ends with 01" is trivial nondeterministically)**.** States $\{q_0, q_1, q_2\}$ (q_2 accept); from q_0, on 0 go to $\{q_0, q_1\}$ (stay or guess this is the last 01's 0), on 1 stay at q_0; $q_1 \xrightarrow{1} q_2$. A DFA for the same language needs 3 states anyway, but for "nth symbol from end is 1" the NFA has $n+1$ states while DFA needs 2^n.

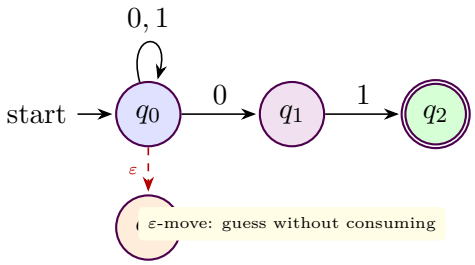


Figure 2.3: NFA for Σ^*01 with an ε-shortcut illustrating nondeterministic branching; acceptance is existential over paths.

2.3 Subset Construction: NFA → DFA

Powerset DFA D simulates *all* NFA possibilities simultaneously: states are subsets of Q.

Theorem 2.1 (Subset construction)**.** For every NFA N with n states there is a DFA D with $\leq 2^n$ states and $L(D) = L(N)$. Construction: $Q_D = \mathcal{P}(Q)$, $q_{0D} = \textsc{Eclose}(\{q_0\})$, $\delta_D(S, a) = \textsc{Eclose}(\bigcup_{p \in S} \delta(p, a))$, $F_D = \{S \mid S \cap F \neq \emptyset\}$.

Proof. Induction on $|w|$: $\hat{\delta}_D(q_{0D}, w) = \hat{\delta}_N(q_0, w)$ as sets. Base ε by Eclose definition. Step wa: $\hat{\delta}_D(S, wa) = \delta_D(\hat{\delta}_D(S, w), a) = \textsc{Eclose}(\bigcup \delta(p, a)) = \hat{\delta}_N(q_0, wa)$. Hence w reaches accept in D iff some NFA path reaches F. □ □

Blow-up is unavoidable in worst case: $L_n = \{w \mid |w| \geq n$ and nth-from-end is $1\}$ has $n + 1$-state NFA but every DFA needs $\geq 2^n$ states (pairwise distinguishable suffixes: Myhill–Nerode preview in Ch. 3). Subset construction is thus tight.

Example 2.4 (Subset on "ends with 01")**.** N states $\{0, 1, 2\}$. Reachable subsets from $\{0\}$: $\{0\} \xrightarrow{0} \{0, 1\}$, $\{0\} \xrightarrow{1} \{0\}$, $\{0, 1\} \xrightarrow{0} \{0, 1\}$, $\{0, 1\} \xrightarrow{1} \{0, 2\}$ accept, $\{0, 2\} \xrightarrow{0} \{0, 1\}$, $\{0, 2\} \xrightarrow{1} \{0\}$—4 reachable of 8 possible.

2.4 Regular Expressions

Regexes are syntax for regular languages.

Definition 2.3 (Regex and $L(R)$)**.** Basis: $\emptyset$, ε, a for $a \in \Sigma$ with $L(\emptyset) = \emptyset$, $L(\varepsilon) = \{\varepsilon\}$, $L(a) = \{a\}$. Inductive: if R_1, R_2 regex then $(R_1 + R_2)$, $(R_1 R_2)$, (R^*) with $L(R_1 + R_2) = L(R_1) \cup L(R_2)$, $L(R_1 R_2) = L(R_1)L(R_2)$, $L(R^*) = L(R)^*$. We write $+$ for union, omit $\cdot$ for concat, $R^+ = RR^*$, $R^? = R + \varepsilon$.

Example 2.5 (Regex shorthands)**.** Over $\{0, 1\}$: $(0 + 1)^* = \Sigma^*$; $(0 + 1)^*01$ = ends with 01; $(0 + 1)^*01(0 + 1)^*$ = contains 01; 0^*1^* = some 0s then some 1s ($\neq (0 + 1)^*$). Note a^* vs. a^+ vs. $a^?$.

2.5 Kleene's Theorem: Regex ≡ NFA ≡ DFA

Theorem 2.2 (Kleene, 1956)**.** L is regular (DFA-recognisable) iff it is denoted by some regex, iff it is recognised by some NFA/ε-NFA. Conversions are effective.

Sketch. DFA⇒NFA trivial (deterministic is special case). Regex⇒ ε-NFA by Thompson: build inductively for $\emptyset, \varepsilon, a$ (2-state fragments) then compose with ε-branches for $+$, chain for $\cdot$, loop for $*$. NFA⇒DFA is subset construction. DFA⇒regex by state elimination: generalise transitions to regex labels and rip out states one by one, replacing $p \xrightarrow{R_1} q \xrightarrow{R_2} r$ plus loop R_3 at q with $p \xrightarrow{R_1 R_3^* R_2} r$ and $p \xrightarrow{R_1 R_3^* R_2 + R_4} r$ if direct R_4 exists. □ □

Example 2.6 (DFA→regex on even 0s)**.** Eliminate q_1 from Fig. 2.1: loop on q_0 labelled $1 + 01^*0$? Instead systematic: start with 2-state GNFA, eliminate q_1: self-loop 1 at q_1, $q_0 \to q_1$ on 0, $q_1 \to q_0$ on 0, so after elimination q_0 gets $1 + 0(1)^*0$? Actually $1^*0(1^*0)^*$? Simpler: regex $1^*(01^*01^*)^*$. Tools automate; key is existence.

Remark 2.1 (Thompson in code). Thompson's inductive step is literally an AST walk:

```
def thompson(node):
    if node.kind=='lit': return fragment(node.char)      # 2 states
    if node.kind=='concat': return concat(thompson(node.left), thompson(node.right))
    if node.kind=='union':  return union(thompson(node.left), thompson(node.right))  #
        epsilon fork
    if node.kind=='star':   return star(thompson(node.child))  # epsilon loop
```

Each combinator adds at most 2 new states and ε-edges, so size is linear in $|R|$. Epsilon-closure then costs $O(|Q| + |\delta|)$ per step.

2.6 Chapter Notes

Rabin and Scott introduced NFAs and subset construction (1959); Kleene linked regexes to automata (1956). Brzozowski derivatives give an alternative regex→DFA. Closure properties make regular languages a Boolean algebra; the product and subset constructions are reused for intersection, complement, and decidability in Ch. 3. For implementation, prefer Thompson + subset lazily (`on-the-fly`) to avoid the 2^n blow-up unless minimised. Aho–Sethi–Ullman (Compilers) gives the industrial Thompson implementation used in `grep`.

2.7 Exercises

E2.1 **DFA table.** Build DFA for $L = \{w \mid w \text{ contains } 010 \text{ as substring}\}$ with 4 states. Give δ table, identify trap, and trace $w = 00101$.

E2.2 **NFA to DFA.** Convert NFA for "ends with 01" via subset construction; list all 4 reachable subsets and draw resulting DFA. Is it already minimal?

E2.3 **Regex to ε-NFA.** Thompson-construct ε-NFA for $(a + b)^*abb$; count states before and after ε-removal, then simulate on $ababb$.

E2.4 **Distinguishing power.** Prove L_n (nth-from-end is 1) needs $n+1$-state NFA but $\geq 2^n$ DFA states: show 2^n suffixes x_i pairwise distinguishable by some z.

E2.5 **State elimination.** Eliminate states on DFA with $Q = \{q_0, q_1, q_2\}$, q_2 accept, transitions $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$, $q_1 \xrightarrow{a} q_1$, $q_0 \xrightarrow{b} q_0$, to obtain regex for $b^*ab^*?a(a + b)^*$ variant; verify on aab.

Chapter 3

Properties of Regular Languages

> *"How do we prove a language is* not *regular?"* — DFAs have finite memory; a language needing unbounded counting cannot be regular. Pumping and Myhill–Nerode make this precise, and minimisation squeezes every DFA to its canonical core.

From zero: we build here, no prior distinguishability assumed. We start from the pigeonhole principle, then state pumping, then the stronger Myhill–Nerode characterisation, then DFA minimisation as partition refinement. Everything proved from definitions in Ch. 2.

Learning Outcomes

After this chapter you will be able to:

(L1) state and apply the pumping lemma for regular languages;

(L2) choose a witness w and pump to prove non-regularity (e.g. 0^n1^n, palindromes);

(L3) state Myhill–Nerode and count equivalence classes to prove (non-)regularity;

(L4) explain closure properties and how they combine with pumping/Myhill–Nerode;

(L5) minimise a DFA via table-filling / partition refinement and argue uniqueness.

3.1 Why Some Languages Need Infinite Memory

A DFA with p states reading a word of length $\geq p$ must repeat a state (pigeonhole). The loop between the two visits can be repeated or deleted—this is the seed of pumping.

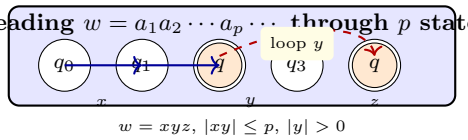


Figure 3.1: Pigeonhole forces a repeat: $w = xyz$ where y loops on the repeated state; $xy^i z$ stays accepted.

3.2 The Pumping Lemma for Regular Languages

Theorem 3.1 (Pumping lemma, regular)**.** If L regular then $\exists p \geq 1$ (pumping length) such that $\forall w \in L$ with $|w| \geq p$, $\exists$ split $w = xyz$ with (i) $|xy| \leq p$, (ii) $|y| > 0$, (iii) $\forall i \geq 0 :\ xy^i z \in L$.

Proof. Take DFA M for L with $p = |Q|$ states. Trace w of length $\geq p$: the $p+1$ prefixes visit $p+1$ states, so some state repeats within first p symbols. Let x lead to first visit, y the loop, z the rest: $|xy| \leq p$, $|y| > 0$, and looping y any i times stays in the same accept/reject status. □ □

Contrapositive use: to show L not regular, pick for each p a long $w \in L$ and show *every* admissible x, y, z fails to pump.

Example 3.1 (Classic $0^n 1^n$ is not regular)**.** $L = \{0^n 1^n \mid n \geq 0\}$. Assume regular with p. Choose $w = 0^p 1^p \in L$, $|w| \geq p$. Any split xyz with $|xy| \leq p$ forces $y = 0^k$, $k > 0$, lying entirely in the 0-block. Then $xy^2 z = 0^{p+k} 1^p \notin L$ (too many 0s), violating (iii). Contradiction. Hence L not regular. Same idea shows $\{0^n 1^n \mid n \geq 1\}$, $\{ww \mid w \in \{0,1\}^*\}$ not regular.

Example 3.2 (Palindromes not regular)**.** $L = \{w \mid w = w^R\}$ over $\{0,1\}$. Take $w = 0^p 1 0^p \in L$ (even length with centre 1). Any y in first p symbols is 0^k, pumping disrupts symmetry: $xy^2 z = 0^{p+k} 1 0^p \notin L$. The crucial point is $|xy| \leq p$ pins y to the left block.

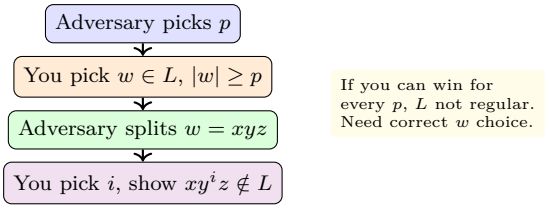


Figure 3.2: The pumping game: existential/universal quantifiers as a game; winning strategy proves non-regularity.

Remark 3.1 (Common pitfalls). Pumping lemma is necessary but not sufficient; there are non-regular languages that still pump (e.g. $\{a^i b^j c^k \mid i = 0 \text{ or } j = k\}$). Use Myhill–Nerode for a complete characterisation. Also w must be chosen wisely: $0^p 1^p$ works, but $0^p 1^p$ with $i = 0$ vs. 2 matters—always exhibit one i that breaks membership.

3.3 Myhill–Nerode: The Exact Criterion

Define $x \sim_L y$ iff $\forall z \in \Sigma^*,\ xz \in L \Leftrightarrow yz \in L$ (indistinguishable extensions). $\sim_L$ is an equivalence; its classes partition Σ^*.

Theorem 3.2 (Myhill–Nerode)**.** L regular iff $\sim_L$ has finitely many classes. Number of classes = number of states of the minimal DFA for L.

Sketch. ($\Rightarrow$) DFA with $|Q|$ states puts strings reaching the same state into the same class: at most $|Q|$ classes. ($\Leftarrow$) Build DFA whose states *are* the classes: $\delta([x], a) = [xa]$, $q_0 = [\varepsilon]$, $F = \{[x] \mid x \in L\}$. Well-defined and minimal because any DFA must separate distinguishable strings. □ □

Example 3.3 (0^n1^n has infinitely many classes)**.** For $x_m = 0^m$ and $x_n = 0^n$ with $m \neq n$, take $z = 1^m$: $x_m z = 0^m 1^m \in L$ but $x_n z = 0^n 1^m \notin L$, so $x_m \not\sim_L x_n$. Hence infinitely many distinct classes $\{0^n\}$; L not regular and no finite DFA exists.

Example 3.4 (When Myhill–Nerode is easier)**.** $L = \{w \mid \#_0(w) = \#_1(w)\}$ (balanced, not 0^*1^*). For $x_k = 0^k$, $z = 1^k$ distinguishes, giving infinite classes—one-line proof vs. tricky pumping. Even $L = \{a^n b^n \mid n \geq 0\} \cup \{b^n a^n\}$ also needs Myhill–Nerode.

Example 3.5 (Code: distinguishability test)**.**

```python
def distinguish(L, x, y, bound=6):
    # brute-force search for separating extension z up to bound
    from itertools import product
    Sigma=['0','1']
    for l in range(bound+1):
        for z in product(Sigma, repeat=l):
            z=''.join(z)
            if (x+z in L) != (y+z in L):
                return z  # z separates x,y
    return None
# distinguish({0n1n}, '00','000') -> '11' separates
```

For regular L, this search terminates by Myhill–Nerode finiteness; for non-regular, it may need unbounded z.

Theorem 3.3 (Regular pumping vs. Myhill–Nerode strength)**.** If $\sim_L$ has p classes then p is a pumping length. Conversely, a language can pump with p yet have infinitely many $\sim_L$-classes (pumping is strictly weaker).

Sketch. If p classes, among $p + 1$ prefixes of any $|w| \geq p$ two fall in same class; their difference loop pumps. Counterexample for converse is the Bar-Hillel language above; pumping holds vacuously for short witnesses but distinguishability still infinite. □ □

3.4 DFA Minimisation

Among all DFAs for L, one is smallest (up to isomorphism). Minimise by merging indistinguishable states.

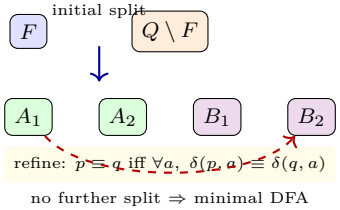


Figure 3.3: Table-filling / partition refinement: start $\{F, Q \setminus F\}$, repeatedly split blocks whose a-successors land in different blocks; remaining blocks are the minimal states.

Algorithm (table-filling, $O(|Q|^2|\Sigma|)$): mark pairs (p, q) with $p \in F \not\ni q$ as distinguishable; iteratively mark (p, q) if $\exists a$ with $(\delta(p, a), \delta(q, a))$ already marked. Unmarked pairs are equivalent—merge them. Hopcroft's $O(|\Sigma||Q| \log |Q|)$ is the fast variant.

Remark 3.2 (Why minimise?). Minimal DFA is the Myhill–Nerode DFA; any larger DFA wastes states that are indistinguishable. Minimisation also gives a decision procedure for equivalence: minimise both DFAs and check isomorphism (up to renaming)—linear after minimisation. Brzozowski's double-reversal trick also yields the minimal DFA and is often simpler to implement than table-filling. In practice, keep DFAs minimal after each product/closure to prevent state explosion.

Example 3.6 (Minimising "ends with 01" DFA)**.** Subset DFA from Ch. 2 had 4 states: $\{0\}, \{0,1\}, \{0,2\}, \{0\}$? Actually 3 reachable non-equivalent. Table-filling finds $\{0\}$ and $\{0,2\}$ distinguishable from others but $\{0\}$ initial and final? Ends with 01 minimal DFA has 3 states (residuals: ε, 0, 01). No two merge, so already minimal. A DFA with an extra dead trap plus duplicate accept can be collapsed to this.

3.5 Closure and Decidability Preview

Regular languages are closed under $\cup, \cap, -, \cdot, *, R$, homomorphism and inverse homomorphism. Each closure has a constructive automaton, yielding decidability: emptiness (F reachable from q_0?), universality ($\overline{L} = \emptyset$?), inclusion ($L_1 \setminus L_2 = \emptyset$?), equivalence ($L_1 \oplus L_2 = \emptyset$?) are all decidable via graph reachability on automata. Finiteness is also decidable via pumping length p: check strings up to p and p–$2p$.

Example 3.7 (Closure in action)**.** To decide if $L = \{0^n 1^n\}$ were regular (it is not), intersect with regular 0^*1^* does nothing; but $L_1 = \{a^m b^n \mid m \neq n\}$ is $a^*b^* \setminus \{a^n b^n\}$; if $\{a^n b^n\}$ were regular its complement in a^*b^* would be too. Using closure we transport non-regularity without repeating pumping. Complement flips F; product gives $\cap$; De Morgan gives $\cup$.

Theorem 3.4 (Decidability of emptiness etc.)**.** Given DFA M, $L(M) = \emptyset$ iff no accept state reachable from q_0 (BFS $O(|Q| + |\delta|)$). Hence universality, inclusion, equivalence are decidable: $L_1 \subseteq L_2$ iff $L_1 \cap \overline{L_2} = \emptyset$; $L_1 = L_2$ iff both inclusions hold.

3.6 Chapter Notes

Pumping lemma folklore (Bar-Hillel); Myhill–Nerode (1957–58) gives the definitive test and minimisation correctness. Brzozowski's minimisation (reverse–determinise–reverse–determinise) is an elegant alternative. Hopcroft's $O(n \log n)$ partition refinement is optimal for minimisation and mirrors DFA learning. Non-closure of decidability beyond regular is the theme of Ch. 7: emptiness stays decidable for CFLs but equivalence does not. For practice, implement table-filling in Python and test on random DFAs; minimal DFAs are unique up to renaming, giving a canonical form for L. Reachability plus minimisation reduces many regular-language questions to graph algorithms.

3.7 Exercises

E3.1 **Pumping warm-up.** Prove $L = \{0^n 1^n 2^n \mid n \geq 0\}$ not regular in two ways: (a) pumping, (b) Myhill–Nerode via 0^n. Which is shorter?

E3.2 **Closure vs. pumping.** Show $L = \{a^m b^n \mid m \neq n\}$ not regular using closure under complement/intersection with a^*b^*. Why does a direct pump with $w = a^p b^p$ need care?

E3.3 **Myhill–Nerode counting.** For $L = \{w \mid w \text{ ends with } 010\}$, compute $|\Sigma^*/\sim_L|$ and build minimal DFA; exhibit distinguishing extensions for each pair of residuals.

E3.4 **Minimisation trace.** Minimise DFA with $Q = \{A, B, C, D, E\}$, $F = \{C\}$, δ given by table (provide). Use table-filling, list marked pairs in order, give quotient DFA and argue minimality via Myhill–Nerode.

E3.5 **Pumping is not sufficient.** Give a non-regular L that satisfies the pumping lemma (hint: $\{a^i b^j c^k \mid i = 0 \text{ or } j = k\} \cup a^*b^*$) and explain why Myhill–Nerode still detects non-regularity (infinitely many $b^j c^k$ classes).

Chapter 4

Context-Free Grammars and Languages

"Regular languages cannot count." — To capture 0^n1^n and nesting (parentheses, `if-else`), we add recursion: grammars whose rules expand a variable into strings of variables and terminals.

From zero: we build here, no grammar assumed. We define CFG from scratch—terminals, variables, productions, derivations—with trees before formalism. Induction on derivations proves language membership. Only Ch. 1–2 required.

Learning Outcomes

After this chapter you will be able to:

(L1) define CFG, derivation, parse tree and language $L(G)$;

(L2) design CFGs for counting and nesting languages (0^n1^n, palindromes, a^nb^m);

(L3) determine ambiguity and eliminate it where possible (or prove inherent);

(L4) convert a CFG to Chomsky normal form (CNF) with proof sketch;

(L5) state and apply the pumping lemma for CFLs to show non-context-freeness.

4.1 Context-Free Grammars

Intuition: a variable is a placeholder for a language; a rule $A \to \alpha$ says "replace A by α". Starting from S, repeatedly replace until only terminals remain.

Definition 4.1 (CFG)**.** A *context-free grammar* is $G = (V, \Sigma, R, S)$ where V variables, Σ terminals ($\Sigma \cap V = \emptyset$), $R \subseteq V \times (V \cup \Sigma)^*$ finite productions $A \to \alpha$, $S \in V$ start. Write $\Rightarrow$ for one-step derivation ($\beta A \gamma \Rightarrow \beta\alpha\gamma$ if $A \to \alpha$), $\Rightarrow^*$ for zero-or-more steps. $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$.

Example 4.1 (The canonical 0^n1^n)**.** G: $S \to 0S1 \mid \varepsilon$. Derivation of 0011: $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$.

Language is $\{0^n1^n \mid n \geq 0\}$—proved by induction: $S \Rightarrow^* 0^n1^n$ for all n (induction on n), and no other strings derivable (induction on derivation length; the first rule applied determines the outer 0/1 pair).

Example 4.2 (Palindromes and nesting)**.** Over $\{0,1\}$: $P \to 0P0 \mid 1P1 \mid 0 \mid 1 \mid \varepsilon$ generates all palindromes. For balanced parentheses with $\Sigma = \{(,)\}$: $S \to SS \mid (S) \mid \varepsilon$. For $a^ib^jc^k$ with $i = j$ or $j = k$ variations, we branch at the top: $S \to AB \mid CD$ with separate sub-grammars—union via new start.

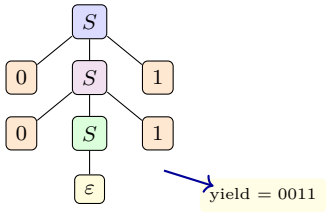


Figure 4.1: Parse tree for 0011 via $S \to 0S1$: recursion nests; frontier (leaves left-to-right) is the derived string. Every derivation corresponds to a tree.

Definition 4.2 (Parse tree, leftmost derivation)**.** A *parse tree* has root S, interior nodes in V, leaves in $\Sigma \cup \{\varepsilon\}$, and children of A spell a production $A \to \alpha$. Yield = concatenation of leaves. A *leftmost* derivation always expands the leftmost variable; it traces a depth-first traversal.

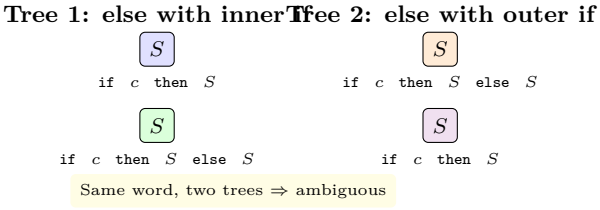


Figure 4.2: The dangling-else ambiguity: one string, two parse trees. Real languages fix this by grammar refactoring or precedence declarations.

Definition 4.3 (Ambiguity)**.** G is *ambiguous* if some $w \in L(G)$ has two distinct parse trees (equivalently two leftmost derivations). L is *inherently ambiguous* if every CFG for L is ambiguous (e.g. $\{a^ib^jc^k \mid i = j \text{ or } j = k\}$).

Example 4.3 (Proving $L(G) = \{0^n1^n\}$ by induction)**.** We show $S \Rightarrow^* 0^n1^n$ by induction on n (base ε, step via $S \to 0S1$). Conversely, if $S \Rightarrow^* w$, induction on derivation length: first step must be $S \Rightarrow 0S1 \Rightarrow^* w$, so $w = 0x1$ with shorter derivation $S \Rightarrow^* x$; by IH $x = 0^{n-1}1^{n-1}$, hence $w = 0^n1^n$. This two-direction induction is the template for all CFG correctness proofs.

Remark 4.1 (Design pattern). For union $L_1 \cup L_2$, new $S \to S_1 \mid S_2$; for concat $S \to S_1S_2$; for star $S \to SS_1 \mid \varepsilon$. These combinators mirror regex operators and preserve context-freeness directly via grammar surgery.

4.2 Chomsky Normal Form

CNF restricts productions to $A \to BC$ or $A \to a$ (plus $S \to \varepsilon$ if needed), enabling $O(n^3)$ parsing (CYK) and pumping proofs.

Theorem 4.1 (CNF conversion)**.** For every CFG G there is G' in CNF with $L(G') = L(G) \setminus \{\varepsilon\}$ (and $S \to \varepsilon$ added iff $\varepsilon \in L(G)$). Transformation is effective: eliminate ε-productions, unit productions, and long right-hand sides.

Sketch. 1. Nullable: compute A with $A \Rightarrow^* \varepsilon$ (fixed point), add all ε-free variants of each production (if $A \to BCD$ with C nullable, add $A \to BD$ etc.). Remove $A \to \varepsilon$ except possibly $S \to \varepsilon$. *2. Units:* if $A \overset{*}{\Rightarrow} B$ via $A \to B$, replace $A \to B$ by $A \to \alpha$ for each $B \to \alpha$ non-unit. *3. Long:* replace $A \to X_1 \cdots X_k$ ($k \geq 3$) by $A \to X_1Y_1$, $Y_1 \to X_2Y_2$, ...; replace terminals a in long bodies by new $T_a \to a$. Each step preserves language. □ □

Example 4.4 (CNF trace)**.** G: $S \to aSb \mid SS \mid \varepsilon$ (balanced a^nb^n concatenations). Nullable $= \{S\}$. After step 1: $S \to aSb \mid ab \mid SS \mid S \mid \varepsilon$; step 2 removes $S \to S$; step 3 breaks aSb into $S \to AT$, $T \to SB$, $A \to a$, $B \to b$. Result has ≤ 2 symbols per body. CYK can now parse ab via AB.

Example 4.5 (Why CNF matters for complexity)**.** Without CNF, a production $A \to BCDE$ could be arbitrarily long, but CNF's binary form bounds branching. Hence any w with $|w| = n$ has a parse tree with $2n-1$ internal nodes (exactly), giving the $2^{|V|}$ pumping bound and the $O(n^3)$ CYK table size—both impossible without length restriction.

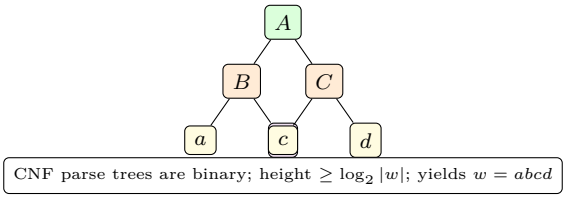


Figure 4.3: In CNF every internal node has exactly two variable children or one terminal; parse trees are binary, so long words force a tall branch that will repeat a variable.

4.3 Pumping Lemma for CFLs

Idea: tall CNF tree for long w has a root-to-leaf path with repeated variable A (pigeonhole on $|V|$); the subtree rooted at the higher A can be pumped.

Theorem 4.2 (Pumping lemma, CFL)**.** If L CFL then $\exists p \geq 1$ (here $p = 2^{|V|}$) such that $\forall w \in L$, $|w| \geq p$, $\exists$ split $w = uvxyz$ with (i) $|vxy| \leq p$, (ii) $|vy| > 0$, (iii) $\forall i \geq 0 :\ uv^ixy^iz \in L$.

Sketch. Take CNF G for $L \setminus \{\varepsilon\}$. For $|w| \geq 2^{|V|}$, any parse tree has height $\geq |V| + 1$, so some variable A repeats on a leaf-to-root path. Let upper $A \Rightarrow^* vAy$ (with v, y possibly empty but not both) and $A \Rightarrow^* x$. Then $S \Rightarrow^* uAy \Rightarrow^* uvAyz \Rightarrow^* uvxyz$ and iterating the A-loop gives uv^ixy^iz. Bound $|vxy| \leq 2^{|V|} = p$ because the subtree depth at the repeat is $\leq |V| + 1$. □ □

Example 4.6 (Showing $\{a^nb^nc^n \mid n \geq 0\}$ not CFL)**.** Assume CFL with p. Take $w = a^pb^pc^p$. Any split $uvxyz$ with $|vxy| \leq p$ touches at most two of the three blocks (pigeonhole on blocks). If vxy lies in a's and b's, pumping changes a's or b's but not c's, breaking n-equality. Formal: if v or y contains mixed symbols a and b, uv^2xy^2z has shape violation $a^*b^*a^*$...; else pumping unbalances counts. Hence not CFL. By closure, $\{a^nb^nc^n\}$ separates CFL from regular.

Example 4.7 (When pumping fails but L still not CFL)**.** $L = \{a^ib^jc^k \mid i \leq j \leq k\}$ is not CFL but pumping alone needs care because $|vxy| \leq p$ can be placed to pump within one block while preserving $i \leq j \leq k$. Instead use Ogden's lemma (marked positions) or closure: intersect with $a^*b^*c^*$ and use stronger condition. This mirrors the regular case where pumping is necessary but not sufficient.

Remark 4.2 (Pumping game for CFLs). As with regular pumping, the CFL game is: adversary picks p, you pick w, adversary splits $w = uvxyz$, you pump. The bound $|vxy| \leq p$ is the new weapon—it forces vxy to be local, so a clever $w = a^p b^p c^p$ spreads over three blocks and any local vxy misses one block. Choose w to maximise this locality trap.

4.4 Closure Properties of CFLs

CFLs are closed under $\cup$, concat, $*$, substitution, homomorphism, inverse homomorphism, reversal, but *not* under $\cap$ or complement. Example: $L_1 = \{a^n b^n c^m\}$, $L_2 = \{a^m b^n c^n\}$ are CFL, but $L_1 \cap L_2 = \{a^n b^n c^n\}$ is not. Hence $\overline{L_1}$ cannot be CFL (otherwise $\cap$ would be via De Morgan and $\cup$). Intersection with a *regular* language *does* stay CFL (product of PDA and DFA in Ch. 5), a useful tool: intersect suspected non-CFL with $a^*b^*c^*$ to isolate the hard core before pumping.

Example 4.8 (CYK preview on CNF)**.** CYK parses $w = a_1 \cdots a_n$ by DP: $T[i, j] = \{A \mid A \Rightarrow^* a_i \cdots a_j\}$. Initialise $T[i, i] = \{A \mid A \to a_i\}$; then $T[i, j] = \bigcup_k \{A \mid A \to BC,\ B \in T[i, k],\ C \in T[k + 1, j]\}$. Accept iff $S \in T[1, n]$. For $w = ab$ with CNF above, $T[1, 1] = \{A\}$, $T[2, 2] = \{B\}$, $T[1, 2]$ contains S via $S \to AB$—$O(n^3|G|)$.

Theorem 4.3 (Decidability for CFLs)**.** Emptiness ($L(G) = \emptyset$?), finiteness, and membership ($w \in L(G)$?) are decidable (reachability on variables and CYK). Universality, equivalence, inclusion, and regularity of a CFL are undecidable (reduce from halting via Ch. 7). Membership via CYK is $O(n^3|G|)$ in CNF; emptiness is linear by marking reachable variables—graph reachability again. Finiteness is also decidable: check for a pumpable variable that derives itself with non-trivial yield and is reachable from S and can derive a terminal string.

4.5 Chapter Notes

Chomsky (1956) introduced CFGs and CNF; Bar-Hillel's CFL pumping (1961) and later Ogden (1968) refined non-membership proofs. CYK parsing ($O(n^3|G|)$) follows from CNF's binary shape and is used in Ch. 5 for PDA equivalence. Inherent ambiguity was shown by Parikh for $\{a^i b^j c^k \mid i = j \text{ or } j = k\}$. Greibach normal form ($A \to a\alpha$, $\alpha \in V^*$) is an alternative that yields top-down parsing and connects to PDAs in Ch. 5. For compilers, LL/LR subsets of CFGs suffice for deterministic parsing; the general CFG problem is $O(n^3)$. The pumping bound $p = 2^{|V|}$ is tight: the language $\{a^{2^n}\}$ needs that many variables in CNF to force a repeat, illustrating exponential succinctness gaps between grammars.

4.6 Exercises

Each exercise reinforces the grammar $\leftrightarrow$ tree $\leftrightarrow$ CNF $\leftrightarrow$ pumping cycle. Exercises E4.4–E4.5 use Ogden or closure with $a^*b^*c^*$ to avoid case analysis traps.

E4.1 **Grammar design.** Give CFG for $L = \{0^i 1^j \mid i \leq j\}$ and for $L = \{w \mid w \text{ has more 0s than 1s}\}$. Prove correctness by induction on derivations for one of them.

E4.2 **Ambiguity test.** Is $S \to S + S \mid S * S \mid (S) \mid a$ ambiguous? Exhibit two trees for $a + a * a$, then refactor with precedence layers $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to (E) \mid a$ and argue unambiguous.

E4.3 **CNF conversion.** Convert $S \to aSb \mid SS \mid \varepsilon$ to CNF step by step: list nullable, eliminate ε, eliminate units, break long bodies. Show parse of $aabb$ in CNF.

E4.4 **CFL pumping.** Use pumping to prove $\{a^n b^n c^n \mid n \geq 0\}$ not CFL; consider cases where vxy straddles blocks vs. lies inside one block, and why $|vxy| \leq p$ forces at most two blocks.

E4.5 **Non-closure.** Prove CFLs not closed under intersection via $L_1 = \{a^n b^n c^m\}$ and $L_2 = \{a^m b^n c^n\}$; deduce non-closure under complement. Give closure under union/concat/$*$ by construction on grammars.

E4.6 **Bonus: Ogden.** State Ogden's lemma (marked positions) and use it to show $\{a^i b^j c^k \mid i \leq j \leq k\}$ not CFL more cleanly.

Chapter 5

Pushdown Automata

"A stack is counting you can pop." — Add a stack to a finite control and 0^n1^n becomes recognisable; add only a stack and you exactly capture the context-free languages.

From zero: we build here, no PDA assumed. We define PDA from scratch—states, input, stack alphabet, push/pop—with pictures of stack height before formalism. CFG↔PDA equivalence is proved by two simulations. Only Ch. 4 required.

Learning Outcomes

After this chapter you will be able to:

(L1) define PDA, instantaneous description and acceptance by final state / empty stack;

(L2) simulate a PDA on 0^n1^n and on palindromes, explaining stack as counter;

(L3) convert CFG to PDA (top-down) and PDA to CFG (triple construction) at high level;

(L4) state the equivalence theorem PDA $\equiv$ CFG and outline both directions;

(L5) explain why deterministic PDAs are strictly weaker and give an example.

5.1 Pushdown Automata: Finite Control + Stack

Intuition: a DFA cannot remember n; a stack can. Push 0s, pop one per 1—if empty at the end, n matched. The stack is LIFO: only the top is visible.

Definition 5.1 (PDA)**.** A *pushdown automaton* is $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where Q finite states, Σ input alphabet, Γ stack alphabet, q_0 start, $Z_0 \in \Gamma$ initial stack symbol, $F \subseteq Q$ accept, and $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$ finite. $(p, \gamma) \in \delta(q, a, X)$ means: in q reading a (or ε) with top X, go to p and replace X by γ (push, pop, or replace).

An *ID* is (q, w, γ): state, remaining input, stack contents (left = top). Step $(q, aw, X\beta) \vdash (p, w, \gamma\beta)$ if $(p, \gamma) \in \delta(q, a, X)$; also ε-moves $(q, w, X\beta) \vdash (p, w, \gamma\beta)$. Acceptance: by *final state* ($q \in F$ after consuming all input, stack anything) or by *empty stack* (stack becomes ε). The two modes are equivalent (add new bottom marker and draining states).

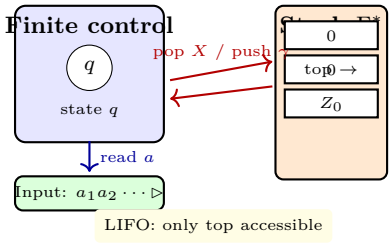


Figure 5.1: PDA: finite control points to input head and stack top; a transition $\delta(q, a, X) \ni (p, \gamma)$ replaces top X by γ while moving the input head if $a \neq \varepsilon$.

Example 5.1 (PDA for $L = \{0^n 1^n \mid n \geq 0\}$ by empty stack)**.** $\Gamma = \{Z_0, 0\}$; push 0 for each 0 input, pop one 0 per 1. Sketch: $\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$, $\delta(q_0, 0, 0) = \{(q_0, 00)\}$, $\delta(q_0, 1, 0) = \{(q_1, \varepsilon)\}$, $\delta(q_1, 1, 0) = \{(q_1, \varepsilon)\}$, $\delta(q_1, \varepsilon, Z_0) = \{(q_f, \varepsilon)\}$. Trace 0011: stack $Z_0 \to 0Z_0 \to 00Z_0 \xrightarrow{1} 0Z_0 \xrightarrow{1} Z_0 \xrightarrow{\varepsilon} \varepsilon$ accept. Non-member 001 blocks with stack $0Z_0 \neq \varepsilon$.

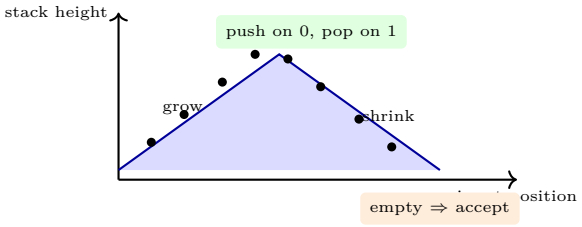


Figure 5.2: Stack height mirrors the count: rises n steps on 0s, falls n steps on 1s; mismatch leaves stack non-empty or blocks on empty pop.

Example 5.2 (Palindromes need nondeterminism)**.** $L = \{ww^R \mid w \in \{0,1\}^*\}$ (even palindromes) PDA: push input until nondeterministically guess the middle, then pop and match. Formally q_0 pushes $0, 1$; ε-move $q_0 \to q_1$ (switch to popping); in q_1, $\delta(q_1, 0, 0) = \{(q_1, \varepsilon)\}$ etc.; $w \in L$ iff some guess of middle position matches. No DPDA recognises all palindromes (deterministic cannot guess centre).

Remark 5.1 (Acceptance modes unified). To convert empty-stack → final-state, add fresh q_f accepting and ε-moves draining stack to q_f; to convert final-state → empty-stack, add bottom marker X_0 and a draining phase that pops everything after entering F. Hence we may assume whichever is convenient in proofs. Both conversions add at most one new state and one stack symbol, so size blow-up is constant.

Example 5.3 (Code: PDA simulator (nondeterministic BFS))**.**

```
def accepts_pda(pda, w):
    from collections import deque
    frontier = deque([(pda.q0, 0, (pda.Z0,))])  # (state, pos, stack tuple top-first)
    visited=set()
    while frontier:
        q,pos,stk = frontier.popleft()
        if pos==len(w) and q in pda.F:  # accept by final state
            return True
        if (q,pos,stk) in visited: continue
        visited.add((q,pos,stk))
        top = stk[0] if stk else None
```

```
        for (a,X), moves in pda.delta.items():
            if X != top: continue
            if a != '' and (pos>=len(w) or w[pos]!=a): continue
            for p,gamma in moves:
                npos = pos+1 if a!='' else pos
                nstk = tuple(gamma)+stk[1:] if gamma else stk[1:]
                frontier.append((p,npos,nstk))
    return False
```

BFS over IDs explores nondeterministic branching; ε-moves keep `pos` unchanged, so bound depth or detect loops via visited.

5.2 Equivalence: CFG $\Leftrightarrow$ PDA

Theorem 5.1 (CFG $\equiv$ PDA)**.** L is context-free iff some PDA recognises L (by final state or empty stack). Conversions are effective.

Intuition for CFG $\to$ PDA (top-down): PDA simulates leftmost derivation on its stack: stack holds the "yet-to-derive" suffix. If top is variable A, nondeterministically replace it by α for some $A \to \alpha$ (ε-move). If top is terminal a, match it against next input symbol and pop on equality. Start with SZ_0? Actually start symbol S on stack above Z_0; accept when stack empties after matching input. This is how LL parsers work.

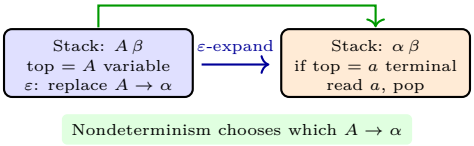


Figure 5.3: CFG→PDA: expand variables via ε-moves; match terminals against input. Every leftmost derivation corresponds to a PDA computation and vice versa.

Intuition for PDA $\to$ CFG (triple construction): variable $[pXq]$ means "from state p with stack top X, PDA can reach q and pop X (net effect: stack height unchanged beyond X)". Productions: if $\delta(p,a,X) \ni (r,\varepsilon)$ then $[pXr] \to a$; if $\delta(p,a,X) \ni (r,YZ)$ then $[pXq] \to a[rYs][sZq]$ for all s,q. Start $S \to [q_0 Z_0 q]$ for all $q \in F$. Induction shows $[pXq] \Rightarrow^* w$ iff $(p,w,X) \vdash^* (q,\varepsilon,\varepsilon)$. Details are standard but lengthy; the idea is the stack discipline forces nested intervals.

Example 5.4 (PDA→CFG on $\{0^n1^n\}$)**.** Triple variables include $[q_0 Z_0 q_f]$, $[q_0 0 q_1]$, etc. From $\delta(q_0,0,0) = (q_0,00)$ we get $[q_0 0 s] \to 0[q_0 0 t][t 0 s]$; from $\delta(q_1,1,0) = (q_1,\varepsilon)$ we get $[q_1 0 q_1] \to 1$. The resulting grammar generates 0^n1^n after eliminating useless variables, essentially recovering $S \to 0S1 \mid \varepsilon$.

Remark 5.2 (Determinism). DPDA: $\delta(q,a,X)$ has at most one move and at most one of $\delta(q,a,X)$, $\delta(q,\varepsilon,X)$ defined (no choice). DPDA $\subsetneq$ PDA = CFG: $L = \{a^n b^n\} \cup \{a^n b^{2n}\}$ is CFL but no DPDA recognises it (needs to decide which block count after reading a's). DPDA languages (DCFLs) are closed under complement but not union. LR grammars correspond to DPDAs.

Example 5.5 (DPDA vs. PDA: $\{wcw^R\}$ is deterministic)**.** With centre marker c, PDA is deterministic: push w until c, then pop and match. Formal DPDA: in q_0, push $0,1$ on $0,1$; on c go to q_1; in q_1, $\delta(q_1,0,0) = (q_1,\varepsilon)$, $\delta(q_1,1,1) = (q_1,\varepsilon)$. Acceptance by final state at bottom Z_0. Removing c (palindromes ww^R) forces nondeterminism—no DPDA exists, proving DCFL $\neq$ CFL.

Example 5.6 (Stack as counter with two symbols)**.** $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$ has PDA that nondeterministically guesses which equality to check: branch 1 pushes a's then pops on b's and ignores c's; branch 2 ignores a's then pushes b's and pops on c's. Either branch suffices, so union is CFL despite no single counter handling both equalities.

5.3 Closure and Decidability for CFLs via PDAs

CFLs are not closed under $\cap$/complement, but $L \cap R$ with R regular *is* CFL (product of PDA and DFA: keep DFA state in PDA control). This preserves decidability of emptiness ($L(G) = \emptyset$? check if S derives a terminal string) and membership (CYK $O(n^3)$), but universality, equivalence, and inclusion become undecidable (via halting, Ch. 7). The triple construction also shows every CFL is accepted by empty stack with a single-state PDA? Actually 2 states suffice.

Theorem 5.2 (Regular intersection)**.** If L CFL and R regular then $L \cap R$ CFL. Construction: $P' = (Q \times Q_R, \Sigma, \Gamma, \delta', (q_0, r_0), Z_0, F \times F_R)$ where $\delta'((q, r), a, X) \ni ((p, \delta_R(r, a)), \gamma)$ for each $(p, \gamma) \in \delta(q, a, X)$. Then $L(P') = L(P) \cap R$.

The construction mirrors the DFA product in Ch. 2; the DFA component is encoded in the finite control, leaving the stack untouched. Hence regular constraints do not add stack power.

Example 5.7 (Using $L \cap R$ to prove non-CFL)**.** $L = \{a^n b^n c^n \mid n \geq 0\}$ is not CFL, but $L_1 = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$ is. Intersect L_1 with regular $a^*b^*c^*$ does nothing; but $L_1 \cap a^*b^*c^*$ already shows why closure matters. For $L = \{ww \mid w \in \{0,1\}^*\}$, intersect with $0^*1^*0^*1^*$ to force shape, then pump.

Remark 5.3 (Why PDAs are nondeterministic by default). Unlike DFAs, deterministic PDAs are strictly weaker. The subset-like construction fails because stacks cannot be subset-combined: two stack contents 00 and 11 cannot be merged into a single "super-stack". Hence we keep PDAs nondeterministic when proving CFG≡PDA.

5.4 Chapter Notes

Oettinger (1961) and Schützenberger proved CFG≡PDA; Ginsburg–Greibach systematised the triple construction. Evey (1963) optimised state count. For parsing, LL corresponds to top-down PDA simulation, LR to bottom-up (handle) simulation; Knuth's LR theorem characterises DCFLs as exactly the deterministic languages. The single-stack restriction is essential: two stacks give Turing power (Ch. 6); one stack plus nondeterminism gives exactly CFLs. Hopcroft–Ullman gives the full triple proof; Sipser presents a streamlined version. Implement the CFG→PDA conversion and test on 0^n1^n to see stack growth.

5.5 Exercises

Nondeterminism, stack invariants, and the two simulations are the testable skills; E5.5 separates DCFL from CFL. Try implementing the wcw^R DPDA deterministically before attempting palindromes nondeterministically. For E5.4, eliminate useless variables: a variable is useful only if reachable from S and can derive a terminal string.

E5.1 **PDA trace.** Simulate the 0^n1^n PDA on 0011 and 0101; list IDs (q, w, γ) step by step and indicate where nondeterminism would branch for palindromes.

E5.2 **Design PDAs.** Give PDA (by final state) for $L = \{0^i1^j \mid i \leq j\}$ and for $L = \{a^nb^nc^m \mid n + m \text{ even}\}$; explain stack invariant "height = excess 0s".

E5.3 **CFG → PDA.** Convert $S \to 0S1 \mid \varepsilon$ to a top-down PDA: list Γ, Z_0, and each δ entry; trace 01 and show the stack content after each expand/match.

E5.4 **PDA → CFG.** Apply triple construction to the 1-state PDA for $\{a^nb^n\}$ with single control q and $\Gamma = \{Z_0, X\}$; simplify to $S \to aSb \mid \varepsilon$ after removing useless variables.

E5.5 **Determinism gap.** Prove no DPDA accepts $L = \{a^nb^n\} \cup \{a^nb^{2n}\}$: assume DPDA exists, show it must commit after a^n without knowing whether b^n or b^{2n} follows, then fool it with a distinguishing continuation.

Chapter 6

Turing Machines and the Church–Turing Thesis

> *"A Turing machine is an idealised laptop whose memory never runs out."* — Finite control plus an unbounded tape models every effective computation; the Church–Turing thesis claims no stronger model exists.

From zero: we build here, no TM assumed. We define TM from scratch—tape, head, transition—with pictures of configurations before formalism. Variants (multi-tape, nondeterministic) are shown equivalent; universal TM is the stored-program computer. Only Ch. 1–5 required.

Learning Outcomes

After this chapter you will be able to:

(L1) define TM, configuration, computation and language $L(M)$ / function computed;

(L2) simulate a TM for 0^n1^n and verify it on examples;

(L3) transform multi-tape and nondeterministic TMs to single-tape deterministic with polynomial/exp overhead;

(L4) state Church–Turing thesis and distinguish thesis from theorem;

(L5) describe the universal TM and its role as interpreter.

6.1 The Turing Machine Model

Intuition: a finite automaton with a tape it can read, write, and move left/right arbitrarily—unbounded memory indexed by integers. At each step the control looks at the current state and tape symbol under the head, writes a symbol, moves L/R, and changes state.

Definition 6.1 (Turing machine)**.** A *(deterministic) TM* is $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where Q finite states, Σ input alphabet ($\sqcup \notin \Sigma$ blank), $\Gamma \supseteq \Sigma \cup \{\sqcup\}$ tape alphabet, $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ partial (undefined = halt), q_0 start, $q_{\text{accept}} \neq q_{\text{reject}}$ halting accept/reject. Tape is infinite in both directions (or right-infinite with left end-marker—equivalent), initially contains $w \in \Sigma^*$ with blanks elsewhere, head at cell 0.

A *configuration* is (q, tape contents, head position), written e.g. uqv where q is just before the head symbol. Computation is the sequence of $\vdash_M$ steps given by δ; M *accepts* w if it reaches q_{accept}, *rejects* if q_{reject} or halts otherwise, *loops* if never halts. $L(M) = \{w \mid M \text{ accepts } w\}$. TMs may also *compute functions*: output is tape contents at halt.

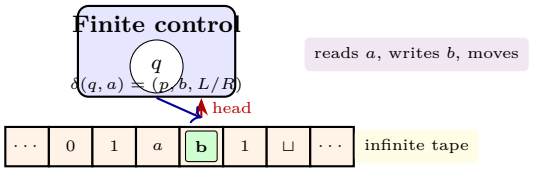


Figure 6.1: TM: control reads tape cell under head, consults δ, writes, moves L/R. The tape is the unbounded memory; the control is finite.

Example 6.1 (TM for $L = \{0^n 1^n \mid n \geq 0\}$)**.** Idea: repeatedly cross off one 0 (mark as X) and one 1 (mark as Y). States: scan right for 0, mark X, go right past X's/Y's to find 1, mark Y, return left to start; accept when no 0 left and no 1 remains unmarked. Transition fragment: $\delta(q_0, 0) = (q_1, X, R)$, $\delta(q_1, 0) = (q_1, 0, R)$, $\delta(q_1, Y) = (q_1, Y, R)$, $\delta(q_1, 1) = (q_2, Y, L)$, $\delta(q_2, Y) = (q_2, Y, L)$, $\delta(q_2, 0) = (q_2, 0, L)$, $\delta(q_2, X) = (q_0, X, R)$; plus checks for malformed 1 before 0 to reject. This TM is total (halts on all inputs)—a decider.

Example 6.2 (Code: TM configuration stepper)**.**

```
def step(tm, config):
    q, tape, head = config  # tape dict pos->symbol, blanks implicit
    a = tape.get(head, '_')  # '_' = blank
    if (q,a) not in tm.delta: return None  # halt
    p,b,dir = tm.delta[(q,a)]
    ntape = dict(tape); ntape[head]=b
    nhead = head+1 if dir=='R' else head-1
    return (p, ntape, nhead)
# BFS simulation of nondeterministic variants uses queue over configs
```

The tape is a sparse map; only visited cells are stored, modelling the infinite tape finitely.

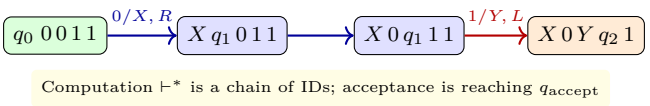


Figure 6.2: Initial IDs for 0011 on the $0^n 1^n$ TM: crossing off $0 \to X$ and $1 \to Y$ iterates until acceptance.

6.2 Variants: Robustness of the Model

The TM is absurdly robust: adding tapes, nondeterminism, two-dimensional tapes, or random access does not change the class of computable languages (only efficiency).

Theorem 6.1 (Multi-tape → single-tape)**.** For every k-tape TM running in time $t(n)$ there is a single-tape TM simulating it in $O(t(n)^2)$ (and $O(t(n) \log t(n))$ with better encoding). Idea: interleave tapes on one tape with

#-separators and dot-marked head positions; to simulate one multi-step, scan whole tape to gather symbols, update, and move dots.

Theorem 6.2 (Nondeterministic → deterministic)**.** For every NTM N there is a DTM D with $L(D) = L(N)$ and time $2^{O(t(n))}$ (breadth-first search over computation tree; branch $\leq b$, depth t, so $\leq b^t$ leaves). Hence nondeterminism does not add computability, only (possibly) exponential speed-up.

DFS would be unsound: a DTM doing depth-first might follow an infinite branch and never explore a finite accepting branch; BFS (or iterative deepening) avoids this by fairness. The exponential blow-up b^t is inherent for naive simulation; smarter simulations (Savitch) achieve $O(t^2)$ space but remain exponential time. This gap is the seed of complexity theory in Ch. 8: is nondeterminism *really* exponential, or can it be simulated polynomially?

Other equivalences: 2-stack PDA ≡ TM, 2-counter machine ≡ TM, λ-calculus ≡ TM, Python/Java ≡ TM (with unbounded memory). This convergence underlies the thesis below.

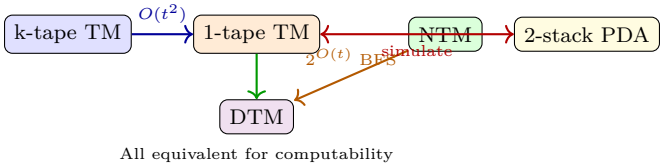


Figure 6.3: TM variants simulate each other with at most polynomial (multi-tape) or exponential (nondeterminism) overhead; computability is invariant.

6.3 Church–Turing Thesis and Encodings

Definition 6.2 (Church–Turing thesis)**.** Every effectively calculable function (by any reasonable physical or combinatorial model) is computable by a Turing machine. This is a *thesis*, not a theorem—it is an empirical claim about the informal notion "effectively calculable", supported by all known models collapsing to TMs.

We fix an encoding $\langle M \rangle$ (e.g. binary) of TM M and $\langle M, w \rangle$ of pair; languages become sets of strings over $\{0, 1\}$, so TMs can take other TMs as input. This self-reference enables diagonalisation (Ch. 7) and the universal TM.

Theorem 6.3 (Universal TM)**.** There exists UTM U such that $U(\langle M, w \rangle) = M(w)$ (with same accept/reject/loop behaviour). Construction: U keeps M's state and tape on its own tapes, looks up δ_M in the encoded table, and steps faithfully. This is the stored-program computer: software as data.

Sketch. U has 3 tapes: input ($\langle M, w \rangle$ never altered), simulated tape of M, and simulated state. Initialise simulated tape to w, state to q_0^M. Loop: scan $\langle M \rangle$ for entry $\delta_M(q, a) = (p, b, D)$, update simulated tape/state, move simulated head. If M halts, U halts correspondingly. □ □

Example 6.3 (UTM in code outline)**.**

```
def UTM(M_code, w):
    q, tape, head = parse_initial(M_code, w)
    table = parse_table(M_code)  # dict (q,a)->(p,b,dir)
    while q not in ('acc','rej'):
        a = tape.get(head, '_')
```

```
        if (q,a) not in table: break  # halt = reject by convention
        p,b,dir = table[(q,a)]
        tape[head]=b; head+= 1 if dir=='R' else -1; q=p
    return q=='acc'
```

UTM is just an interpreter: the table is data, not hard-wired logic.

6.3.1 Countability and Functions

$\langle M \rangle$ ranges over $\{0,1\}^*$, but not every string is a valid code—we can decide validity syntactically (check δ format). Hence TMs are countable, but languages (subsets of $\{0,1\}^*$) are uncountable (Ch. 1), so most languages are not even recognisable. A TM may also *compute* $f : \Sigma^* \to \Sigma^*$ by halting with $f(w)$ on tape. This aligns with Python functions that may loop; total functions correspond to deciders.

Example 6.4 (TM computing $f(w) = w^R$)**.** 3-tape TM: copy input to tape 2, then write back reversed onto tape 1 (move tape 2 head to end, then step left while writing right on tape 1). This function TM always halts, so f is computable and total.

6.4 Decidability Preview and Complexity Hint

M is a *decider* if it halts on all inputs; L is *decidable* (recursive) if some decider recognises it; *recognisable* (r.e.) if some TM recognises it (may loop on $w \notin L$). Total TMs (deciders) are the effective procedures of interest. Next chapter shows 0^n1^n is decidable, $a^nb^nc^n$ is decidable, but $\{\langle M, w\rangle \mid M \text{ halts on } w\}$ is *not* decidable—the first non-trivial limit of computation.

Example 6.5 (Decidable vs. recognisable)**.** $L_{\text{reg}} = \{\langle M\rangle \mid L(M) \text{ is regular}\}$ is *not* decidable nor recognisable; its complement is also not recognisable—a non-r.e. language (Ch. 7, Rice). By contrast, $A_{\text{DFA}} = \{\langle D, w\rangle \mid D \text{ DFA accepts } w\}$ is decidable (simulate DFA $O(|w|)$), and $A_{\text{CFG}} = \{\langle G, w\rangle \mid G \text{ CFG derives } w\}$ is decidable via CYK $O(|w|^3)$.

Remark 6.1 (Complexity hint). Variant simulations incur overhead: multi-tape to single-tape $O(t^2)$, NTM to DTM $2^{O(t)}$. These overheads *define* P vs. NP in Ch. 8: does nondeterminism give superpolynomial speed-up? For now, computability already separates decidable from undecidable; Ch. 8 refines decidable into feasible (P) vs. infeasible.

6.5 Chapter Notes

Turing (1936) and Church (1936) defined computation; Kleene, Post, Gödel gave equivalent models. The tape $\sqcup$ vs. X/Y marking economy is modern presentation (Sipser). Multi-tape equivalence is standard (Hartmanis–Stearns). UTM is the intellectual ancestor of von Neumann architecture: program and data share the same memory. Sipser and Hopcroft–Ullman give the full multi-tape simulation with $O(t \log t)$ via Hennie–Stearns; the simple $O(t^2)$ interleaving suffices for computability and P vs. NP overhead analysis in Ch. 8. For a physical analogy, the TM tape is your laptop's SSD, the head is the read/write pointer, and the control is the CPU—unbounded disk is the idealisation that makes halting undecidable.

6.6 Exercises

Exercises E6.1–E6.2 test TM programming by marking; E6.3–E6.5 test encodings and the UTM interpreter pattern that underlies Ch. 7 diagonalisation. For E6.4, choose a concrete binary code: q in $\lceil \log |Q| \rceil$ bits, a in $\lceil \log |\Gamma| \rceil$ bits, direction in 1 bit; sum over $|\delta|$ entries.

E6.1 **TM trace.** Give δ for TM deciding $\{0^{2n} \mid n \geq 0\}$ (even zeros) by sweeping and crossing off pairs; trace 000 (reject) and 0000 (accept) as ID sequences.

E6.2 **Multi-tape simulation.** Describe 2-tape TM for $\{0^n 1^n\}$ that copies input to tape 2 then matches; simulate conversion to 1-tape and count overhead $O(n^2)$ steps.

E6.3 **NTM to DTM.** NTM for $\{ww \mid w \in \{0,1\}^*\}$ guesses middle nondeterministically; show DTM BFS explores $\leq 2^n$ branches and why depth-first would be unsound (may follow infinite branch forever).

E6.4 **Encoding.** Fix binary encoding $\langle M \rangle$ with states numbered $0..|Q|-1$; encode δ as list of (q, a, p, b, dir). Count bits for $|Q| = 5$, $|\Gamma| = 4$ and argue encoding size is $O(|Q||\Gamma| \log(|Q||\Gamma|))$.

E6.5 **UTM correctness.** Prove $U(\langle M, w \rangle)$ accepts iff $M(w)$ accepts, loops iff M loops, by induction on computation length: U's simulated config after t steps equals M's config after t steps.

E6.6 **Bonus: 2-counter.** Sketch how 2 stacks simulate a TM tape (push left/right halves) and how 2 counters simulate a stack (Gödel numbering), concluding 2-counter $\equiv$ TM.

Chapter 7

Decidability, Undecidability and Reductions

"There are problems no algorithm can solve." — Diagonalisation and reductions prove the halting problem undecidable, then transport that impossibility to every non-trivial property of programs via Rice's theorem.

From zero: we build here, no undecidability assumed. We define decidable vs. recognisable from Ch. 6 TMs, prove halting undecidable by diagonalisation, then build mapping reductions and Rice. Only TM definitions and encodings $\langle M, w \rangle$ required.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish decidable (recursive), recognisable (r.e.) and co-recognisable;

(L2) state and prove $HALT$ undecidable via diagonalisation;

(L3) construct mapping reductions $A \leq_m B$ and use them to transfer (un)decidability;

(L4) state Rice's theorem and apply it to semantic properties ($L(M) = \emptyset$, regularity, etc.);

(L5) classify A_{TM}, E_{TM}, EQ_{TM} as undecidable/recognisable with reductions.

7.1 Decidable vs. Recognisable

Recall from Ch. 6: L is *decidable* if some TM decider halts on *all* w and $w \in L \iff$ accept. L is *recognisable* (r.e.) if some TM M halts-accepts exactly on $w \in L$ (may loop otherwise). Decidable $\Rightarrow$ recognisable; converse fails strictly.

Definition 7.1 (Complements and co-r.e.)**.** $\overline{L} = \Sigma^* \setminus L$. L is *co-recognisable* if $\overline{L}$ is recognisable. L decidable iff L and $\overline{L}$ both recognisable (run both recognisers in parallel—dovetail—and one must halt).

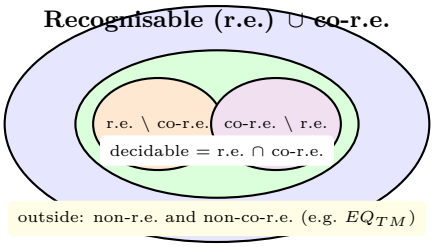


Figure 7.1: Decidable is the intersection of r.e. and co-r.e.; $HALT$ is r.e. but not decidable; EQ_{TM} is neither. Diagonal languages live outside.

Example 7.1 (Decidable warm-ups)**.** $A_{DFA} = \{\langle D, w\rangle \mid D \text{ DFA accepts } w\}$ decidable (simulate DFA). $A_{CFG} = \{\langle G, w\rangle \mid G \text{ derives } w\}$ decidable (CYK). $E_{DFA} = \{\langle D\rangle \mid L(D) = \emptyset\}$ decidable (graph reachability). By contrast, $A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$ is recognisable (simulate M; accept if it does) but *not* decidable—the gap starts with TMs.

Remark 7.1 (Recognisable = enumerable). We will use "r.e. = recognisable = enumerable" interchangeably. An enumerator is a TM that prints L's strings (in any order, with repetitions allowed) without input; its existence is equivalent to a recogniser. Dovetailing gives the enumerator for recognisable L explicitly.

Example 7.2 (Code: dovetail decider from two recognisers)**.**

```python
def decider_for_L(w, M_L, M_comp):
    # M_L recognises L, M_comp recognises complement
    for t in range(1000000):  # dovetail by time bound t
        if simulates_accepts(M_L, w, t): return True
        if simulates_accepts(M_comp, w, t): return False
# Halts iff w in L or w not in L, i.e. iff both are r.e., yielding decidability
```

If both L and $\overline{L}$ are r.e., this interleaving always halts, giving a decider; hence decidable = r.e. ∩ co-r.e.

7.2 The Halting Problem is Undecidable

$HALT = \{\langle M, w\rangle \mid M \text{ halts on } w\}$; A_{TM} is similar but requires *acceptance*. HALT is r.e. (simulate, accept if halt) but undecidable.

Theorem 7.1 (Turing, 1936 — Halting undecidable)**.** $HALT$ (equivalently A_{TM}) is undecidable: no TM decider H exists with $H(\langle M, w\rangle) =$ accept iff M halts on w.

Diagonalisation. Assume decider H for HALT. Build D on input $\langle M\rangle$: run $H(\langle M, \langle M\rangle\rangle)$; if H says "halts" then D loops forever (e.g. `while True: pass`), else D halts. So $D(\langle M\rangle)$ halts iff $M(\langle M\rangle)$ loops. Now run D on $\langle D\rangle$: $D(\langle D\rangle)$ halts iff $D(\langle D\rangle)$ loops—contradiction. Hence H cannot exist. For A_{TM}, similar but D rejects vs. accepts. □ □

Corollary 7.1. $HALT$ is recognisable but not decidable; $\overline{HALT}$ is not recognisable. Hence decidability $\subsetneq$ recognisability.

The diagonalisation shows HALT's complement cannot be enumerated: no TM even lists all non-halting pairs. This is stronger than "no decider"—it separates r.e. from co-r.e.

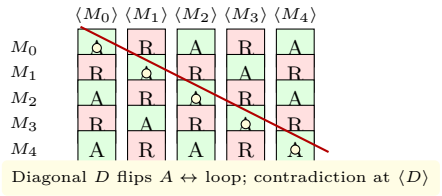


Figure 7.2: Diagonal table: row i is M_i's behaviour on $\langle M_j \rangle$; D flips the diagonal, so D differs from every M_i—no decider for HALT can exist.

7.3 Mapping Reductions

Reductions transfer undecidability: "if I could decide B, I could decide A".

Definition 7.2 (Mapping reduction)**.** $A \leq_m B$ (reducible) if there is computable $f : \Sigma^* \to \Sigma^*$ with $w \in A \iff f(w) \in B$. f is total computable (always halts) by a TM. If $A \leq_m B$ and B decidable then A decidable (compose f with decider for B); contrapositive: A undecidable $\Rightarrow$ B undecidable. Similarly for recognisability: B r.e. $\Rightarrow$ A r.e.

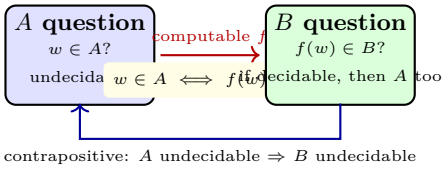


Figure 7.3: Mapping reduction: computable f maps instances of A to instances of B preserving membership; decidability flows forward, undecidability flows backward.

Example 7.3 (HALT $\leq_m E_{TM}$? No, but $A_{TM} \leq_m \overline{E_{TM}}$)**.** $E_{TM} = \{\langle M \rangle \mid L(M) = \emptyset\}$ is undecidable. Reduction from A_{TM}: given $\langle M, w \rangle$, build M' that on x: if $x \neq w$ reject, else simulate M on w and accept iff M accepts. Then $L(M') = \{w\}$ if M accepts w, else $\emptyset$. So $\langle M, w \rangle \in A_{TM} \iff \langle M' \rangle \notin E_{TM}$. Hence $A_{TM} \leq_m \overline{E_{TM}}$; since A_{TM} undecidable and r.e., $\overline{E_{TM}}$ is r.e. but E_{TM} is co-r.e. complete, neither decidable.

Example 7.4 (Code: builder for E_{TM} reduction)**.**

```
def f_A_to_notE(code_M, w):
    # return code of M' that isolates w
    def M_prime(x):
        if x != w: return False
        return simulate(M_code=code_M, input=w)  # accept iff M accepts w
    return encode(M_prime)
# Then w in A_TM iff f(w) not in E_TM
```

The builder is computable: we output the source code of M' syntactically, not by running M.

7.4 Rice's Theorem: Every Non-Trivial Semantic Property is Undecidable

A *semantic* property depends only on $L(M)$, not on code shape or step count. Non-trivial = some TMs have it, some don't.

Theorem 7.2 (Rice, 1953)**.** Let $\mathcal{P} \subseteq \mathcal{P}(\Sigma^*)$ be a set of languages. $L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \in \mathcal{P}\}$ is decidable iff $\mathcal{P}$ is trivial ($\emptyset$ or all r.e. languages). In particular, any non-trivial semantic property of $L(M)$ is undecidable.

Sketch. Assume non-trivial, $\emptyset \notin \mathcal{P}$ w.l.o.g. (otherwise complement). Pick M_0 with $L(M_0) \in \mathcal{P}$. Reduction from A_{TM}: given $\langle M, w\rangle$, build M' that on x: simulate M on w (without touching x); if M accepts w, then run M_0 on x and accept iff M_0 does. If M never accepts, M' loops forever on all x, so $L(M') = \emptyset \notin \mathcal{P}$; if M accepts, $L(M') = L(M_0) \in \mathcal{P}$. Hence $\langle M, w\rangle \in A_{TM} \iff \langle M'\rangle \in L_{\mathcal{P}}$, so $A_{TM} \leq_m L_{\mathcal{P}}$ and $L_{\mathcal{P}}$ undecidable. □ □

Example 7.5 (Rice in action)**.** "$L(M)$ is regular", "$L(M)$ finite", "M halts on ε", "$L(M) = \Sigma^*$" are all undecidable (non-trivial semantic). By contrast, "M has 5 states" is *decidable* (syntactic, inspect code)—Rice does not apply.

7.5 Catalogue: $E_{TM}, EQ_{TM}, REG_{TM}$

We classify canonical problems:

- A_{TM} r.e. but not decidable (as HALT).
- $E_{TM} = \{\langle M\rangle \mid L(M) = \emptyset\}$ co-r.e. but not r.e. (complement is r.e. via $A_{TM} \leq_m \overline{E_{TM}}$); hence undecidable.
- $EQ_{TM} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)\}$ neither r.e. nor co-r.e. (needs both).
- $REG_{TM} = \{\langle M\rangle \mid L(M) \text{ regular}\}$ undecidable by Rice, and neither r.e. nor co-r.e.
- $\overline{A_{TM}}$ not r.e.; dovetail shows decidable = r.e. ∩ co-r.e.

The picture in Fig. 7.1 locates each; reductions justify the inclusions.

7.5.1 Dovetailing: enumerating r.e. languages

If L r.e. via M, the strings in L can be *enumerated* by dovetailing: stage t simulates M on first t strings for t steps each, outputting those that accept within t. This enumerator lists L without repetition; conversely an enumerator gives a recogniser. Dovetailing is the algorithmic picture of "interleaving infinitely many simulations" that makes decidable = r.e. ∩ co-r.e. constructive.

7.6 Chapter Notes

Gödel's incompleteness and Turing's halting are twin diagonalisations. Post and Kleene classified r.e. degrees; Rice's theorem (1953) subsumes countless undecidability proofs in one shot. For practice, always try Rice first for semantic $L(M)$ properties; if syntactic (states, transitions), Rice does not apply. Mapping reductions $\leq_m$ are many-one; stronger Turing reductions ($\leq_T$) allow adaptive queries but are not needed for Rice. The arithmetical hierarchy (Σ_1 = r.e., Π_1 = co-r.e., Δ_1 = decidable) organises these classes; EQ_{TM} is Π_2-complete.

7.7 Exercises

Rice gives one-line undecidability for semantic properties; mapping reductions handle syntactic or mixed properties and completeness arguments. For converse, recall: if $A \leq_m B$ and B r.e. then A r.e.; contrapositive: A not r.e. $\Rightarrow$ B not r.e. (used for EQ_{TM}). E7.4–E7.5 together show the decidable $\subsetneq$ r.e. hierarchy has at least four distinct levels.

E7.1 **Diagonal warm-up.** Reprove $HALT$ undecidable by assuming H decides it and deriving contradiction with $D(\langle D \rangle)$; explain why D must loop (not just reject) to foil deciders that may expect reject vs. loop.

E7.2 **Mapping reduction: $HALT \leq_m E_{TM}$?** Give f that maps $\langle M, w \rangle$ to $\langle M' \rangle$ with $L(M') = \emptyset$ iff M does not halt? Correct the common mistake: halting vs. acceptance matters; produce f for $A_{TM} \leq_m \overline{E_{TM}}$ explicitly.

E7.3 **Rice application.** Use Rice to prove $L = \{\langle M \rangle \mid |L(M)| \geq 3\}$ undecidable, and $L' = \{\langle M \rangle \mid M \text{ has 10 states}\}$ decidable; explain why Rice applies to L but not L'.

E7.4 **Co-r.e. complete.** Show $\overline{A_{TM}}$ is not r.e.: assume it were, then both A_{TM} and $\overline{A_{TM}}$ r.e. would make A_{TM} decidable—contradiction. Deduce E_{TM} co-r.e. via complement reduction.

E7.5 **Neither r.e. nor co-r.e.** Prove EQ_{TM} neither r.e. nor co-r.e.: reduce A_{TM} to $\overline{EQ}$ and $\overline{A_{TM}}$ to EQ via suitable f that builds M_1, M_2 with $L(M_1) = L(M_2)$ conditioned on halting.

Chapter 8

Complexity: P, NP and NP-Completeness

"Can we solve it quickly, or only verify quickly?" — P vs. NP separates feasible from merely checkable; Cook–Levin shows SAT is the hardest checkable problem, and reductions propagate hardness to CLIQUE, 3SAT, and beyond.

From zero: we build here, no complexity assumed. We define P and NP from scratch—deterministic vs. certificate verifiers, time as TM steps—with pictures of exponential vs. polynomial growth before formalism. Cook–Levin and SAT→CLIQUE are proved in outline; coping strategies close the book. Only TMs and reductions required.

Learning Outcomes

After this chapter you will be able to:

(L1) define $TIME(t(n))$, P, NP via verifiers and via NTMs, and explain the two NP views;

(L2) prove $P \subseteq NP$ and know why $P \neq NP$ is open;

(L3) state Cook–Levin: SAT is NP-complete, with proof idea (tableau);

(L4) give a polynomial-time reduction $SAT \leq_p 3SAT \leq_p CLIQUE$ and verify it;

(L5) list coping strategies for NP-complete problems (approximation, FPT, heuristics).

8.1 Time Complexity and P

Fix a (deterministic, single-tape) TM M. Its *time* $time_M(n) = \max_{|w|=n}$ steps to halt (worst case). $TIME(t(n)) = \{L \mid \exists \text{ DTM } M,\ L(M) = L,\ time_M(n) = O(t(n))\}$. Polynomial time is feasible at scale: n^3 stays usable when 2^n does not.

Definition 8.1 (P)**.** $P = \bigcup_{k\geq 1} TIME(n^k)$: languages decidable by a DTM in polynomial time (worst case). P is robust: multi-tape $O(t^2)$, encoding changes, etc. give polynomials, so P is model-invariant (Church–Turing

+ polynomial thesis).

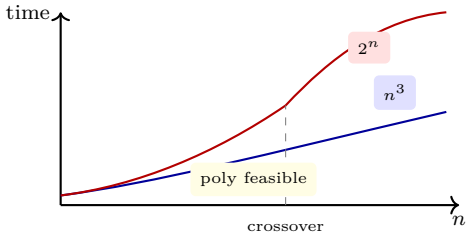


Figure 8.1: Polynomial n^k vs. exponential 2^n: beyond a modest n, exponential dominates regardless of constants; P captures the feasible side.

Example 8.1 (Problems in P)**.** A_{DFA} is in P ($O(n)$ simulation), $REACHABILITY$ ($s \to t$ in digraph) is in P (BFS $O(n+m)$), $RELPRIME$ (gcd) in P (Euclid $O(\log n)$), and even linear programming is in P (Khachiyan 1979). Edges $TIME$ depends on model but P does not.

8.2 NP: Certificates and Nondeterminism

Intuition: a Sudoku is hard to solve but easy to *check* given a filled board—the board is a certificate. NP formalises "checkable in polynomial time".

Definition 8.2 (Verifier view of NP)**.** A *verifier* for L is a DTM V with $L = \{w \mid \exists c,\ |c| \le poly(|w|)$ and $V(w,c) =$ accept$\}$ and V runs in $poly(|w|)$ time. Then $NP = \{L \mid L$ has a poly-time verifier$\}$.

Definition 8.3 (NTM view of NP)**.** $NP = \{L \mid \exists$ NTM $N,\ L(N) = L,\ time_N(n) = O(n^k)\}$ for some k, where $time_N(n)$ is the height of the computation tree (longest branch). Branching is existential: $w \in L$ iff *some* branch accepts within n^k steps.

Theorem 8.1 (Equivalence of views)**.** Verifier $\iff$ NTM. Given verifier V, NTM guesses c nondeterministically ($|c| \le n^k$ possibilities) then runs V; given NTM N, certificate is the sequence of nondeterministic choices leading to accept ($O(n^k)$ bits), verified by simulating N deterministically along those choices.

Example 8.2 (NP examples)**.** $SAT = \{\langle\phi\rangle \mid \phi$ CNF satisfiable$\}$ is in NP: c is assignment, V checks clauses in $O(|\phi|)$. $CLIQUE = \{\langle G,k\rangle \mid G$ has k-clique$\}$ is in NP: c is k-subset, V checks all $\binom{k}{2}$ edges. $HAMPATH$ (Hamiltonian path) also in NP: c is permutation, V checks edges. All are believed not in P.

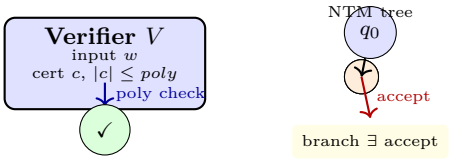


Figure 8.2: Two views of NP: poly-time verifier with short certificate vs. poly-time NTM with existential branching; they are equivalent via "certificate = nondeterministic choices".

Clearly $P \subseteq NP$ (take $c = \varepsilon$, or NTM is DTM). Whether $P = NP$ (can every checkable problem be solved feasibly?) is open and Clay Millennium.

8.3 Polynomial Reductions and NP-Completeness

Reductions now must be *polynomial-time* ($\leq_p$), not just computable.

Definition 8.4 ($\leq_p$, NP-hard, NP-complete)**.** $A \leq_p B$ if computable f with $w \in A \iff f(w) \in B$ and f runs in $poly(|w|)$ time (hence $|f(w)| \leq poly(|w|)$). B is *NP-hard* if $\forall A \in NP$, $A \leq_p B$. B is *NP-complete* if $B \in NP$ and B is NP-hard. If any NP-complete B is in P, then $P = NP$.

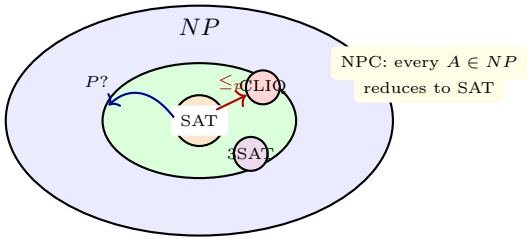


Figure 8.3: NP-complete problems sit at the top of NP: every NP language reduces to them in polynomial time; they stand or fall together for $P = NP$.

8.4 Cook–Levin Theorem: SAT is NP-Complete

Theorem 8.2 (Cook 1971, Levin 1973)**.** SAT (and $3SAT$ with 3 literals per clause) is NP-complete.

Idea — tableau. Given NTM N for $L \in NP$ running in n^k time, on input w build formula $\phi_{N,w}$ whose satisfying assignment encodes an accepting *computation tableau* of N on w: $n^k \times n^k$ grid of tape cells, head positions, and states over time. Variables $x_{i,j,s}$ mean "cell (i, j) at step i holds symbol s". Clauses enforce: (a) each cell has exactly one symbol, (b) initial row is $q_0 w$, (c) each row follows from previous via δ, (d) some row contains q_{accept}. Construction is $O(n^{2k})$ time, poly in $|w|$, and $\phi_{N,w}$ satisfiable iff N accepts w. Hence $L \leq_p SAT$. Since $SAT \in NP$, SAT is NP-complete; $3SAT$ follows by splitting long clauses with fresh variables (e.g. $(a_1 \vee \cdots \vee a_k) \rightsquigarrow (a_1 \vee a_2 \vee z_1) \wedge (\overline{z_1} \vee a_3 \vee z_2) \wedge \cdots$). □ □

Example 8.3 (Tableau size)**.** For $n = 10$, $k = 2$, tableau is $100 \times 100 = 10^4$ cells, each with $|\Gamma| + |Q|$ Boolean variables (≈ 20), so $\approx 2 \times 10^5$ variables and $O(n^{2k})$ clauses—polynomial but large; the theorem is existential, not practical. The clause-splitting for $3SAT$ adds $O(k)$ fresh z's per long clause.

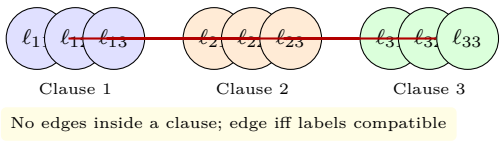
8.5 More NP-Complete Problems: CLIQUE

We propagate completeness by chaining reductions.

Theorem 8.3 (CLIQUE is NP-complete)**.** $3SAT \leq_p CLIQUE$, so CLIQUE is NP-complete.

Sketch. Given 3CNF $\phi = C_1 \wedge \cdots \wedge C_m$ with $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$, build graph G: $3m$ vertices $v_{i,j}$ labelled ℓ_{ij}. Edge $v_{i,j}$–$v_{i',j'}$ iff $i \neq i'$ and labels not contradictory (x vs. $\overline{x}$). Ask for $k = m$-clique. If ϕ satisfiable, pick one true literal per clause—its m vertices are pairwise non-contradictory and cross-clause, so a clique; conversely, a m-clique picks one vertex per clause (clique cannot have two from same C_i since intra-cluster non-edges), and

its labels give a consistent assignment satisfying every clause. Map is $O(m^2)$ time. Since $3SAT$ is NP-complete, so is CLIQUE. □ □



If $\phi = (x \vee y \vee \overline{z}) \wedge (\overline{x} \vee z \vee w) \wedge \cdots$, a 3-clique touching each triangle corresponds to a satisfying choice.

8.6 Coping with NP-Completeness

If $P \neq NP$, NP-complete problems have no polynomial algorithm. Practical coping:

- **Approximation:** for optimisation (e.g. MIN-VERTEX-COVER within $2\times$ via maximal matching), trade optimality for poly time and prove ratio.
- **Fixed-parameter tractability (FPT):** $O(f(k)\,poly(n))$ for parameter k (e.g. VERTEX-COVER $O(2^k n)$ when k = cover size small). Kernelisation shrinks instance.
- **Heuristics & solvers:** SAT solvers (CDCL), ILP, branch-and-bound exploit structure despite worst-case hardness; many real instances are easy.
- **Average case / random:** hardness is worst-case; random 3SAT has phase transition at clause/variable ≈ 4.3.

Recognising NP-completeness saves wasted effort on exact poly algorithms and directs you to the right compromise.

Example 8.4 (Code: brute-force vs. verifier for CLIQUE).

```python
import itertools
def has_clique_bruteforce(G,k):
    for S in itertools.combinations(G.nodes, k):  # exponential
        if all(G.has_edge(u,v) for u,v in itertools.combinations(S,2)):
            return True
    return False
def verify_clique(G,k, cert):
    # cert = k nodes, verifier runs O(k^2)
    if len(cert)!=k or len(set(cert))!=k: return False
    return all(G.has_edge(u,v) for u,v in itertools.combinations(cert,2))
```

Brute force is $O(n^k)$; verifier is $O(k^2)$—the P vs. NP gap in code.

8.7 Chapter Notes

Garey–Johnson is the NP-completeness catalogue; Arora–Barak gives modern treatment with PCP. Cook's original reduction used tableau; Karp (1972) listed 21 NP-complete problems including CLIQUE, HAMPATH,

SUBSET-SUM. For P vs. NP, see Fortnow's survey; relativisation and natural proofs explain why simple diagonalisation fails, motivating circuit and proof complexity approaches.

8.8 Exercises

E8.1 **Verifier vs. NTM.** Give verifier for $HAMPATH$ and convert to NTM: certificate is permutation of n vertices, verifier checks each consecutive pair is an edge in $O(n^2)$. Count certificate bits $O(n \log n)$.

E8.2 **Cook–Levin size.** For NTM with $|\Gamma| = 3$, $|Q| = 10$, $t(n) = n^2$, bound variables and clauses in $\phi_{N,w}$ as polynomials in n; why does poly size matter for $\leq_p$?

E8.3 $SAT \to 3SAT$**.** Reduce clause $(x_1 \vee x_2 \vee x_3 \vee x_4)$ to two 3-clauses with fresh z: $(x_1 \vee x_2 \vee z) \wedge (\overline{z} \vee x_3 \vee x_4)$; prove equisatisfiable and generalise to k-clauses with $k - 3$ fresh variables.

E8.4 **CLIQUE reduction trace.** For $\phi = (x \vee y \vee \overline{z}) \wedge (\overline{x} \vee z \vee w)$, build G with 9 vertices (3 per clause), list edges, exhibit 2-clique vs. 3-clique and read off satisfying assignment $x = 1, y = 0, \ldots$; show unsatisfiable ϕ yields no 3-clique.

E8.5 **Coping.** Show 2-approximation for MIN-VERTEX-COVER: take maximal matching M, return endpoints of M; prove $|M| \leq OPT \leq 2|M|$ and $O(n + m)$ time. Why does this not imply $P = NP$?